# STREAMING DE DADOS PARA A WEB

## Material de Estudo Aprofundado

*Java & Python | Conceitos, Arquitetura e Exemplos Práticos*

**Conteúdo Abordado**

- HTTP Chunked Transfer Encoding
- Long Polling
- Server-Sent Events (SSE)
- WebSockets
- gRPC Streaming
- Reactive Streams (Project Reactor)
- Comparação e Casos de Uso

# Sumário

Sumário ..... 2
1. Introdução ao Streaming de Dados ..... 4
1.1 Motivações para o Uso de Streaming ..... 4
1.2 Modelo de Comunicação: Comparação Básica ..... 4
1.3 Fundamentos Técnicos ..... 5
1.3.1 TCP e Sockets ..... 5
1.3.2 HTTP/1.1 e Conexões Persistentes ..... 5
1.3.3 HTTP/2 e Multiplexação ..... 5
2. HTTP Chunked Transfer Encoding ..... 7
2.1 Funcionamento do Protocolo ..... 7
2.2 Implementação em Java com Spring Boot ..... 7
2.3 Implementação em Python com Flask ..... 9
2.4 Consumindo Chunked Streams no Cliente ..... 10
3. Long Polling ..... 12
3.1 Ciclo de Vida de uma Requisição Long Poll ..... 12
3.2 Implementação em Java com Spring WebFlux (Reativo) ..... 12
3.3 Implementação em Python com FastAPI ..... 14
4. Server-Sent Events (SSE) ..... 16
4.1 Formato do Protocolo SSE ..... 16
4.2 Reconexão Automática ..... 17
4.3 Implementação em Java com Spring WebFlux ..... 17
4.4 Implementação em Python com FastAPI ..... 18
4.5 Consumo SSE no Navegador ..... 19
5. WebSockets ..... 21
5.1 Handshake e Upgrade de Protocolo ..... 21
5.2 Frames WebSocket ..... 21
5.3 Implementação em Java com Spring WebSocket ..... 22
5.4 Implementação em Python com FastAPI (WebSockets) ..... 23
6. gRPC Streaming ..... 25
6.1 Modalidades de Streaming no gRPC ..... 25
6.2 Definição do Contrato (.proto) ..... 25
6.3 Implementação em Java ..... 26
6.4 Implementação em Python com grpcio ..... 27
7. Reactive Streams e Programação Reativa ..... 30
7.1 Componentes da Especificação Reactive Streams ..... 30

7.2 Backpressure: O Problema Central .....30
7.3 Project Reactor: Mono e Flux .....30
7.4 Integração com Spring WebFlux para Streaming HTTP .....32
7.5 Reactive Streams em Python com RxPY e Async Generators.....33
8. Comparação Aprofundada e Guia de Escolha .....35
8.1 Tabela Comparativa Completa .....35
8.2 Guia de Decisão por Cenário.....35
Dashboards e Monitoramento em Tempo Real.....35
Chat e Aplicações Colaborativas .....35
Comunicação entre Microserviços .....36
Exportação de Grandes Volumes de Dados .....36
Notificações Push Simples.....36
8.3 Padrões Arquiteturais para Streaming Escalável .....36
8.3.1 Fan-out com Message Broker .....36
8.3.2 Reconexão com Estado via Last-Event-ID .....37
9. Considerações para Ambientes de Produção .....39
9.1 Segurança.....39
9.2 Gerenciamento de Recursos .....39
9.3 Configuração de Infraestrutura .....39
9.4 Testes de Streaming.....40
10. Conclusão.....42

# 1. Introdução ao Streaming de Dados

1. Introdução ao Streaming de Dados

Em sistemas web tradicionais, a comunicação entre cliente e servidor segue o modelo requisição-resposta (request-response): o cliente envia uma requisição, o servidor processa e retorna uma resposta completa. Esse paradigma, embora simples, apresenta limitações significativas em cenários que exigem transmissão contínua ou em tempo real de informações.

O streaming de dados representa uma abordagem distinta: em vez de aguardar o processamento completo de um recurso para então transmiti-lo, os dados são enviados em fragmentos à medida que se tornam disponíveis. Essa técnica reduz a latência percebida, permite o consumo incremental de grandes volumes de dados e viabiliza comunicação bidirecional e em tempo real.

## 1.1 Motivações para o Uso de Streaming

As principais motivações para adoção de streaming em aplicações web são:

- Redução de latência: o cliente começa a processar dados antes do término da transmissão completa.
- Eficiência de memória: não é necessário armazenar toda a carga útil em memória antes do envio.
- Comunicação em tempo real: eventos do servidor podem ser entregues ao cliente imediatamente ao ocorrerem.
- Suporte a grandes payloads: arquivos e fluxos de dados volumosos podem ser transmitidos sem timeout.
- Processamento progressivo: o cliente pode renderizar ou processar dados parciais (ex.: carregamento progressivo de imagens, resultados de busca).

## 1.2 Modelo de Comunicação: Comparação Básica

O diagrama conceitual a seguir ilustra as diferenças entre os principais modelos de comunicação:

| Modelo | Direção | Conexão | Latência | Caso de Uso Típico |
|---|---|---|---|---|
| Request-Response | Unidirecional | Por requisição | Alta | APIs REST tradicionais |
| Long Polling | Unidirecional | Mantida até resposta | Média | Notificações simples |
| Server-Sent Events | Servidor → Cliente | Persistente | Baixa | Feeds, notificações |
| WebSocket | Bidirecional | Persistente | Muito baixa | Chat, jogos, colaboração |
| HTTP/2 Streaming | Bidirecional | Multiplexada | Baixa | APIs modernas |
| gRPC Streaming | Bidirecional | Persistente | Muito baixa | Microserviços, IoT |

## 1.3 Fundamentos Técnicos

Para compreender adequadamente as técnicas de streaming, é necessário conhecer alguns conceitos de rede subjacentes:

### 1.3.1 TCP e Sockets

O protocolo TCP (Transmission Control Protocol) provê comunicação orientada a fluxo e confiável. Uma conexão TCP é um canal bidirecional de bytes; o HTTP é construído sobre TCP e adiciona semântica de requisição e resposta. O WebSocket, por sua vez, reutiliza a conexão TCP de forma transparente após um handshake HTTP inicial.

### 1.3.2 HTTP/1.1 e Conexões Persistentes

O HTTP/1.1 introduziu conexões keep-alive (persistentes), que permitem múltiplas requisições sobre a mesma conexão TCP. Isso reduz a sobrecarga de estabelecimento de conexão e habilita técnicas como chunked transfer encoding para streaming de respostas.

### 1.3.3 HTTP/2 e Multiplexação

O HTTP/2 introduziu multiplexação de streams, permitindo que múltiplas requisições e respostas trafeguem simultaneamente sobre uma única conexão TCP. Cada stream possui um identificador único e pode ser controlado independentemente, o que é fundamental para gRPC e streaming eficiente de respostas.

> **Nota sobre HTTP/3**
>
> O HTTP/3 substitui o TCP pelo protocolo QUIC (baseado em UDP), eliminando o bloqueio de cabeçalho de linha (head-of-line blocking) e melhorando o desempenho em redes com perdas. Os princípios de streaming permanecem similares, mas com melhor eficiência de transporte.

## 2. HTTP Chunked Transfer Encoding

# 2. HTTP Chunked Transfer Encoding

O Chunked Transfer Encoding é o mecanismo mais básico de streaming sobre HTTP/1.1. Introduzido na RFC 7230, ele permite que o servidor inicie a transmissão de uma resposta antes de conhecer seu tamanho total, enviando o corpo da resposta em fragmentos (chunks).

## 2.1 Funcionamento do Protocolo

Quando o servidor utiliza chunked transfer encoding, o cabeçalho Transfer-Encoding: chunked é incluído na resposta, e o Content-Length é omitido. O corpo da mensagem é composto por uma sequência de chunks, cada um precedido por seu tamanho em hexadecimal, seguido de CRLF, o dado em si, e outro CRLF. O fim do stream é indicado por um chunk de tamanho zero.

Estrutura de um chunk individual:

```
  HTTP Wire Format
<tamanho-em-hex>\r\n
<dados>\r\n

// Exemplo concreto de resposta HTTP chunked:
HTTP/1.1 200 OK
Transfer-Encoding: chunked
Content-Type: text/plain

7\r\n
Mozilla\r\n
9\r\n
Developer\r\n
7\r\n
Network\r\n
0\r\n
\r\n   // chunk zero = fim do stream
```

## 2.2 Implementação em Java com Spring Boot

No ecossistema Spring, o StreamingResponseBody permite que o servidor escreva diretamente no OutputStream da resposta, habilitando chunked transfer encoding de forma transparente:

```
  Java — Spring Boot StreamingResponseBody
// pom.xml: spring-boot-starter-web
```

```
import org.springframework.web.bind.annotation.*;
import
org.springframework.web.servlet.mvc.method.annotation.StreamingResponseBody;
import org.springframework.http.ResponseEntity;
import org.springframework.http.MediaType;
import java.io.*;
import java.nio.charset.StandardCharsets;

@RestController
@RequestMapping("/api/streaming")
public class ChunkedController {

    /**
     * Endpoint que transmite linhas de um arquivo grande em chunks.
     * O cliente começa a receber dados imediatamente, sem aguardar
     * o processamento completo do arquivo.
     */
    @GetMapping(value = "/file", produces = MediaType.TEXT_PLAIN_VALUE)
    public ResponseEntity<StreamingResponseBody> streamLargeFile() {

        StreamingResponseBody stream = outputStream -> {
            try (BufferedWriter writer = new BufferedWriter(
                    new OutputStreamWriter(outputStream,
StandardCharsets.UTF_8))) {

                // Simula processamento de 1000 registros
                for (int i = 1; i <= 1000; i++) {
                    writer.write("Registro #" + i + ": dado_" + i + "\n");
                    writer.flush(); // Força envio do chunk imediatamente

                    // Simula latência de processamento
                    Thread.sleep(10);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        };

        return ResponseEntity.ok()
            .contentType(MediaType.TEXT_PLAIN)
            .body(stream);
    }

    /**
     * Streaming de dados JSON com Jackson.
     * Útil para exportar grandes conjuntos de dados sem
     * carregar tudo em memória.
     */
    @GetMapping(value = "/json-export", produces =
MediaType.APPLICATION_JSON_VALUE)
    public ResponseEntity<StreamingResponseBody> streamJsonExport(
            @RequestParam(defaultValue = "5000") int count) {

        StreamingResponseBody stream = outputStream -> {
            try (PrintWriter writer = new PrintWriter(
                    new BufferedWriter(new
OutputStreamWriter(outputStream)))) {
```

```
                writer.print("[");
                writer.flush();

                for (int i = 0; i < count; i++) {
                    if (i > 0) { writer.print(","); }
                    writer.printf('{"id":%d,"nome":"item_%d","valor":%.2f}',
                        i, i, Math.random() * 1000);
                    writer.flush();
                }

                writer.print("]");
                writer.flush();
            }
        };

        return ResponseEntity.ok().body(stream);
    }
}
```

## 2.3 Implementação em Python com Flask

No Flask, o streaming é habilitado retornando um gerador Python a partir da view function, encapsulado em um objeto Response:

```
  Python — Flask Chunked Streaming
# pip install flask

from flask import Flask, Response, stream_with_context
import json, time, math

app = Flask(__name__)

def gerar_registros(total: int):
    """
    Gerador Python que produz registros um a um.
    Cada yield corresponde a um chunk na resposta HTTP.
    """
    for i in range(1, total + 1):
        registro = {
            "id": i,
            "nome": f"item_{i}",
            "valor": round(math.sin(i) * 100, 2)
        }
        # Separador NDJSON (Newline-Delimited JSON)
        yield json.dumps(registro) + "\n"
        time.sleep(0.005)  # simula latência de BD


@app.route("/api/streaming/export")
def stream_export():
    """
    Retorna 10.000 registros como NDJSON via chunked transfer.
```

```
    O cliente pode processar linha a linha sem buffering total.
    """
    def gerar():
        yield from gerar_registros(10_000)

    return Response(
        stream_with_context(gerar()),
        mimetype="application/x-ndjson",
        headers={"X-Accel-Buffering": "no"}  # Desabilita buffer do Nginx
    )


@app.route("/api/streaming/texto")
def stream_texto():
    """
    Streaming de texto progressivo — padrão de LLMs como o Claude.
    """
    palavras = [
        "Este ", "é ", "um ", "exemplo ", "de ",
        "streaming ", "de ", "texto ", "progressivo."
    ]

    def gerar():
        for palavra in palavras:
            yield palavra
            time.sleep(0.15)

    return Response(gerar(), mimetype="text/plain")


if __name__ == "__main__":
    app.run(debug=True, threaded=True)
```

## 2.4 Consumindo Chunked Streams no Cliente

No lado cliente (navegador), a Fetch API provê suporte nativo a chunked streaming via ReadableStream:

```
  JavaScript — Consumo via Fetch API
// JavaScript (navegador ou Node.js)
async function consumirChunkedStream(url) {
    const response = await fetch(url);
    const reader = response.body.getReader();
    const decoder = new TextDecoder();

    let buffer = "";

    while (true) {
        const { done, value } = await reader.read();
        if (done) break;

        buffer += decoder.decode(value, { stream: true });
```

```
        // Processa linhas completas (para NDJSON)
        const linhas = buffer.split("\n");
        buffer = linhas.pop(); // mantém linha incompleta no buffer

        for (const linha of linhas) {
            if (linha.trim()) {
                const obj = JSON.parse(linha);
                console.log("Recebido:", obj);
            }
        }
    }
}

consumirChunkedStream("/api/streaming/export");
```

**Atenção: Buffering Intermediário**

Proxies reversos como Nginx e servidores de CDN podem bufferizar respostas antes de encaminhá-las ao cliente, anulando os benefícios do streaming. Para desabilitar o buffering no Nginx, utilize o cabeçalho X-Accel-Buffering: no ou configure proxy_buffering off na localização correspondente.

# 3. Long Polling

## 3. Long Polling

Long Polling é uma técnica que simula comunicação em tempo real sobre HTTP convencional. O cliente envia uma requisição que o servidor mantém aberta (pendente) até que um evento ocorra ou um timeout seja atingido. Quando o servidor tem dados para enviar, responde à requisição pendente; o cliente, ao receber a resposta, imediatamente envia uma nova requisição.

### 3.1 Ciclo de Vida de uma Requisição Long Poll

O fluxo de uma sessão long polling pode ser descrito nos seguintes passos:

1. Cliente envia GET /events?lastId=42 ao servidor.
2. O servidor verifica se há eventos novos desde o ID 42.
3. Se houver eventos: retorna imediatamente com os dados.
4. Se não houver eventos: mantém a requisição suspensa (não responde).
5. Quando um evento ocorre ou o timeout expira: o servidor responde.
6. O cliente processa a resposta e retorna imediatamente ao passo 1.

**Características do Long Polling**

Long polling não requer suporte especial no servidor (funciona sobre HTTP/1.0) e é compatível com proxies e firewalls conservadores. Contudo, gera overhead de reconexão constante e não é eficiente para alta frequência de eventos. É adequado para cenários de baixa a média frequência onde SSE ou WebSocket não estejam disponíveis.

### 3.2 Implementação em Java com Spring WebFlux (Reativo)

O Spring WebFlux com DeferredResult ou Mono permite implementar long polling de forma não bloqueante, sem ocupar uma thread por requisição:

Java — Long Polling com Spring WebFlux

```
// pom.xml: spring-boot-starter-webflux

import org.springframework.web.bind.annotation.*;
import org.springframework.http.ResponseEntity;
import reactor.core.publisher.Mono;
import java.time.Duration;
```

```
import java.util.concurrent.*;
import java.util.*;

@RestController
@RequestMapping("/api/longpoll")
public class LongPollController {

    // Fila thread-safe de eventos pendentes
    private final BlockingQueue<String> eventQueue = new
LinkedBlockingQueue<>();

    /**
     * Endpoint de long poll: aguarda até 30s por um novo evento.
     * Retorna 204 (sem conteúdo) se timeout expirar sem eventos.
     */
    @GetMapping("/events")
    public Mono<ResponseEntity<String>> pollForEvent() {
        return Mono
            .fromCallable(() -> eventQueue.poll(30, TimeUnit.SECONDS))
            .subscribeOn(reactor.core.scheduler.Schedulers.boundedElastic())
            .map(event -> event != null
                ? ResponseEntity.ok(event)
                : ResponseEntity.<String>noContent().build()
            );
    }

    /**
     * Endpoint para publicar um evento (simula fonte de dados externa).
     */
    @PostMapping("/publish")
    public ResponseEntity<Void> publishEvent(@RequestBody String payload) {
        eventQueue.offer(payload);
        return ResponseEntity.accepted().build();
    }
}

// ─── Cliente Long Poll em Java (HttpClient) ──────────────────────

import java.net.http.*;
import java.net.URI;

public class LongPollClient {
    private static final HttpClient client = HttpClient.newHttpClient();

    public static void main(String[] args) throws Exception {
        while (true) {
            HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create("http://localhost:8080/api/longpoll/events"))
                .timeout(Duration.ofSeconds(35))
                .GET().build();

            HttpResponse<String> response = client.send(
                request, HttpResponse.BodyHandlers.ofString());

            if (response.statusCode() == 200) {
                System.out.println("Evento recebido: " + response.body());
            }
            // Reconecta imediatamente (sem delay)
```

```
        }
    }
}
```

## 3.3 Implementação em Python com FastAPI

```
  Python — Long Polling com FastAPI
# pip install fastapi uvicorn

from fastapi import FastAPI
from fastapi.responses import JSONResponse
import asyncio
from typing import Optional

app = FastAPI()

# Fila assíncrona de eventos
event_queue: asyncio.Queue = asyncio.Queue()


@app.get("/api/longpoll/events")
async def poll_for_event(timeout: int = 30):
    """
    Aguarda por um evento por até `timeout` segundos.
    Retorna o evento ou 204 se o timeout expirar.
    """
    try:
        evento = await asyncio.wait_for(
            event_queue.get(),
            timeout=timeout
        )
        return JSONResponse(content={"evento": evento})
    except asyncio.TimeoutError:
        return JSONResponse(content=None, status_code=204)


@app.post("/api/longpoll/publish")
async def publish_event(payload: dict):
    """Publica um evento na fila."""
    await event_queue.put(payload)
    return {"status": "publicado"}


# ─── Cliente Long Poll em Python ──────────────────────────────────
# pip install httpx

import httpx, asyncio

async def long_poll_client():
    async with httpx.AsyncClient(timeout=35) as client:
        while True:
            response = await client.get(
                "http://localhost:8000/api/longpoll/events",
                params={"timeout": 30}
            )
            if response.status_code == 200:
```

```
                print("Evento:", response.json())
            # Reconecta imediatamente

asyncio.run(long_poll_client())
```

4. Server-Sent Events (SSE)

# 4. Server-Sent Events (SSE)

Server-Sent Events (SSE) é um padrão W3C que define um protocolo de comunicação unidirecional sobre HTTP, no qual o servidor envia eventos ao cliente de forma contínua. O cliente, tipicamente um navegador, utiliza a interface EventSource para gerenciar a conexão, incluindo reconexão automática e rastreamento de estado via identificadores de eventos.

## 4.1 Formato do Protocolo SSE

O servidor responde com Content-Type: text/event-stream. Os eventos são delimitados por linhas em branco e compõem-se de campos prefixados por nome:

```
SSE Wire Format
// Estrutura de um evento SSE completo
id: 42
event: temperatura
data: {"sensor": "sala-01", "graus": 23.5}
retry: 3000

// Evento sem tipo explícito (event padrão = "message")
id: 43
data: {"mensagem": "Alerta: temperatura elevada"}

// Comentário (mantém a conexão viva)
: ping

// Dado multi-linha (cada linha "data:" é concatenada com \n)
data: linha 1
data: linha 2
data: linha 3
```

Os campos disponíveis no protocolo SSE são:

- id: Identificador do evento. Permite ao cliente rastrear o último evento recebido e solicitar reenvio após reconexão via cabeçalho Last-Event-ID.
- event: Tipo do evento (string arbitrária). O cliente pode registrar listeners específicos por tipo.
- data: Carga útil do evento. Pode ser texto simples ou JSON serializado.
- retry: Intervalo em milissegundos que o cliente deve aguardar antes de reconectar.

## 4.2 Reconexão Automática

Um diferencial importante do SSE é a reconexão automática gerenciada pelo browser. Ao detectar que a conexão foi encerrada, o EventSource reconecta automaticamente, enviando o cabeçalho Last-Event-ID com o ID do último evento processado, permitindo que o servidor retransmita eventos perdidos.

> **SSE vs WebSocket**
> SSE é mais simples de implementar e mais adequado quando a comunicação é predominantemente servidor → cliente (feeds de dados, notificações, progresso de tarefas). Por utilizar HTTP convencional, atravessa proxies e firewalls sem configuração adicional. WebSocket é preferível quando comunicação bidirecional em tempo real é necessária.

## 4.3 Implementação em Java com Spring WebFlux

O Spring WebFlux oferece suporte nativo a SSE via SseEmitter (Spring MVC) ou Flux com MediaType.TEXT_EVENT_STREAM (WebFlux):

Java — SSE com Spring WebFlux

```java
// ─── Spring WebFlux (reativo) ──────────────────────────────────────

import org.springframework.http.MediaType;
import org.springframework.http.codec.ServerSentEvent;
import org.springframework.web.bind.annotation.*;
import reactor.core.publisher.Flux;
import java.time.*;
import java.util.concurrent.atomic.AtomicLong;

@RestController
@RequestMapping("/api/sse")
public class SseController {

    private final AtomicLong counter = new AtomicLong(0);

    /**
     * Stream de eventos de temperatura de sensores.
     * Emite um evento a cada 1 segundo, indefinidamente.
     */
    @GetMapping(value = "/temperatura", produces =
MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<ServerSentEvent<String>> streamTemperatura() {
        return Flux.interval(Duration.ofSeconds(1))
            .map(tick -> {
                long id = counter.incrementAndGet();
                double temp = 20.0 + Math.sin(tick) * 5.0;

                return ServerSentEvent.<String>builder()
                    .id(String.valueOf(id))
                    .event("temperatura")
```

```
                    .data(String.format(
                        '{"sensor":"S01","graus":%.1f,"ts":"%s"}',
                        temp, Instant.now()))
                    .comment("heartbeat")
                    .retry(Duration.ofSeconds(3))
                    .build();
            });
    }

    /**
     * Notificações multicast: todos os clientes conectados
     * recebem o mesmo evento quando publicado.
     */
    private final reactor.core.publisher.Sinks.Many<String> sink =

reactor.core.publisher.Sinks.many().multicast().onBackpressureBuffer();

    @GetMapping(value = "/notificacoes", produces =
MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<ServerSentEvent<String>> streamNotificacoes() {
        return sink.asFlux()
            .map(msg -> ServerSentEvent.<String>builder()
                .event("notificacao")
                .data(msg)
                .build());
    }

    @PostMapping("/notificacoes/publicar")
    public void publicar(@RequestBody String mensagem) {
        sink.tryEmitNext(mensagem);
    }
}
```

## 4.4 Implementação em Python com FastAPI

```
  Python — SSE com FastAPI e sse-starlette
# pip install fastapi uvicorn sse-starlette

from fastapi import FastAPI, Request
from sse_starlette.sse import EventSourceResponse
import asyncio, json, time, math
from datetime import datetime

app = FastAPI()


async def gerador_sensores(request: Request):
    """
    Gerador assíncrono de eventos SSE para dados de sensores.
    Verifica a cada iteracao se o cliente ainda esta conectado.
    """
    event_id = 0
    while True:
        # Verificar desconexão do cliente
        if await request.is_disconnected():
            print("Cliente desconectado, encerrando stream.")
```

```
            break

        event_id += 1
        temperatura = 20.0 + math.sin(event_id * 0.1) * 5.0

        yield {
            "id": str(event_id),
            "event": "temperatura",
            "data": json.dumps({
                "sensor": "S01",
                "graus": round(temperatura, 1),
                "ts": datetime.utcnow().isoformat()
            }),
            "retry": 3000  # ms
        }

        await asyncio.sleep(1)


@app.get("/api/sse/temperatura")
async def sse_temperatura(request: Request):
    """
    Endpoint SSE para dados de temperatura.
    Retorna Content-Type: text/event-stream automaticamente.
    """
    return EventSourceResponse(gerador_sensores(request))


# ─── Cliente SSE em Python ──────────────────────────────────────────
# pip install sseclient-py requests

import sseclient, requests

def consumir_sse(url: str):
    """Consome um stream SSE linha a linha."""
    with requests.get(url, stream=True) as response:
        client = sseclient.SSEClient(response)
        for event in client.events():
            print(f"ID: {event.id} | Tipo: {event.event}")
            dados = json.loads(event.data)
            print(f"  Temperatura: {dados['graus']}°C")

consumir_sse("http://localhost:8000/api/sse/temperatura")
```

## 4.5 Consumo SSE no Navegador

```
  JavaScript — EventSource API
// JavaScript — EventSource API (nativa nos navegadores)
const source = new EventSource("/api/sse/temperatura");

// Listener genérico para eventos "message" (sem tipo definido)
source.onmessage = (event) => {
    const dados = JSON.parse(event.data);
    document.getElementById("temp").textContent = dados.graus + "°C";
};
```

```
// Listener para tipo específico
source.addEventListener("temperatura", (event) => {
    console.log("Leitura:", JSON.parse(event.data));
});

// Tratamento de erros e reconexão
source.onerror = (error) => {
    if (source.readyState === EventSource.CLOSED) {
        console.log("Conexão fechada permanentemente.");
    } else {
        // EventSource reconecta automaticamente (estado CONNECTING)
        console.log("Tentando reconectar...");
    }
};

// Encerrar o stream quando necessário
// source.close();
```

5. WebSockets

# 5. WebSockets

O protocolo WebSocket (RFC 6455) estabelece um canal de comunicação full-duplex e persistente sobre uma única conexão TCP. Diferentemente do HTTP, após o handshake inicial, tanto o cliente quanto o servidor podem enviar mensagens a qualquer momento, sem a necessidade de uma requisição prévia.

## 5.1 Handshake e Upgrade de Protocolo

A conexão WebSocket é iniciada por uma requisição HTTP de upgrade. O cliente envia um cabeçalho Upgrade: websocket e o servidor responde com 101 Switching Protocols:

```
WebSocket Handshake
// Requisição HTTP de upgrade (cliente → servidor)
GET /ws/chat HTTP/1.1
Host: exemplo.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13

// Resposta do servidor (101 Switching Protocols)
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=

// A partir daqui, o protocolo HTTP não é mais utilizado.
// Frames WebSocket trafegam diretamente sobre TCP.
```

## 5.2 Frames WebSocket

O protocolo WebSocket define frames como unidade de transmissão. Um frame contém: um bit FIN (indica se é o frame final da mensagem), um opcode (0x1 = texto, 0x2 = binário, 0x8 = close, 0x9 = ping, 0xA = pong), um bit de mascaramento, o tamanho do payload e os dados.

**Mensagens vs Frames**

> Uma mensagem WebSocket pode ser dividida em múltiplos frames (fragmentação). O receptor deve reassemblar os frames com FIN=0 até receber o frame final com FIN=1. A maioria das bibliotecas gerencia a fragmentação automaticamente.

## 5.3 Implementação em Java com Spring WebSocket

**Java — WebSocket com Spring Boot**

```java
// pom.xml: spring-boot-starter-websocket

import org.springframework.context.annotation.*;
import org.springframework.web.socket.*;
import org.springframework.web.socket.config.annotation.*;
import org.springframework.web.socket.handler.TextWebSocketHandler;
import java.util.concurrent.*;

// ─── Configuração do WebSocket ──────────────────────────────────────
@Configuration
@EnableWebSocket
public class WebSocketConfig implements WebSocketConfigurer {
    @Override
    public void registerWebSocketHandlers(WebSocketHandlerRegistry registry)
{
        registry.addHandler(new ChatWebSocketHandler(), "/ws/chat")
                .setAllowedOrigins("*");
    }
}

// ─── Handler do WebSocket ───────────────────────────────────────────
public class ChatWebSocketHandler extends TextWebSocketHandler {

    // Sessões ativas (thread-safe)
    private final CopyOnWriteArrayList<WebSocketSession> sessions =
        new CopyOnWriteArrayList<>();

    @Override
    public void afterConnectionEstablished(WebSocketSession session) {
        sessions.add(session);
        System.out.println("Nova conexão: " + session.getId());
    }

    @Override
    protected void handleTextMessage(WebSocketSession session,
                                     TextMessage message) throws Exception {
        String payload = message.getPayload();
        System.out.println("[" + session.getId() + "]: " + payload);

        // Broadcast para todos os clientes conectados
        TextMessage resposta = new TextMessage("Broadcast: " + payload);
        for (WebSocketSession s : sessions) {
            if (s.isOpen()) s.sendMessage(resposta);
        }
    }

    @Override
    public void afterConnectionClosed(WebSocketSession session,
```

```
                                        CloseStatus status) {
        sessions.remove(session);
        System.out.println("Desconectado: " + session.getId());
    }

    @Override
    public void handleTransportError(WebSocketSession session,
                                     Throwable exception) {
        System.err.println("Erro: " + exception.getMessage());
    }
}
```

## 5.4 Implementação em Python com FastAPI (WebSockets)

```
  Python — WebSocket com FastAPI
# pip install fastapi uvicorn websockets

from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from typing import List
import json

app = FastAPI()


class GerenciadorConexoes:
    """Gerencia múltiplas conexões WebSocket ativas."""

    def __init__(self):
        self.conexoes_ativas: List[WebSocket] = []

    async def conectar(self, ws: WebSocket):
        await ws.accept()
        self.conexoes_ativas.append(ws)

    def desconectar(self, ws: WebSocket):
        self.conexoes_ativas.remove(ws)

    async def broadcast(self, mensagem: dict):
        """Envia para todos os clientes conectados."""
        for conexao in self.conexoes_ativas:
            await conexao.send_json(mensagem)


gerenciador = GerenciadorConexoes()


@app.websocket("/ws/chat/{usuario}")
async def endpoint_chat(websocket: WebSocket, usuario: str):
    await gerenciador.conectar(websocket)

    await gerenciador.broadcast({
        "tipo": "sistema",
        "mensagem": f"{usuario} entrou no chat."
    })
```

```
    try:
        while True:
            dados = await websocket.receive_json()
            await gerenciador.broadcast({
                "tipo": "mensagem",
                "usuario": usuario,
                "texto": dados.get("texto", ""),
            })
    except WebSocketDisconnect:
        gerenciador.desconectar(websocket)
        await gerenciador.broadcast({
            "tipo": "sistema",
            "mensagem": f"{usuario} saiu do chat."
        })


# ─── Cliente WebSocket em Python ──────────────────────────────────────
# pip install websockets

import asyncio, websockets, json

async def cliente_chat():
    uri = "ws://localhost:8000/ws/chat/alice"
    async with websockets.connect(uri) as ws:
        # Tarefa de recebimento em paralelo
        async def receber():
            async for mensagem in ws:
                print("Recebido:", json.loads(mensagem))

        receber_task = asyncio.create_task(receber())

        # Envia mensagens do terminal
        for texto in ["Olá!", "Como vai?", "Até logo!"]:
            await ws.send(json.dumps({"texto": texto}))
            await asyncio.sleep(1)

        receber_task.cancel()

asyncio.run(cliente_chat())
```

**Heartbeat (Ping/Pong)**

Para manter conexões WebSocket ativas através de proxies e firewalls com timeout agressivo, implemente um mecanismo de ping/pong. O cliente (ou servidor) envia frames Ping periodicamente; o receptor deve responder com Pong automaticamente (a maioria das bibliotecas faz isso). Configure o intervalo em 20-30 segundos para infraestruturas corporativas.

# 6. gRPC Streaming

# 6. gRPC Streaming

gRPC é um framework de chamada de procedimento remoto (RPC) desenvolvido pelo Google, construído sobre HTTP/2 e Protocol Buffers (protobuf). Além de RPCs simples, o gRPC suporta quatro modalidades de comunicação, três das quais envolvem streaming.

## 6.1 Modalidades de Streaming no gRPC

| Modalidade | Cliente → Servidor | Servidor → Cliente | Descrição |
|---|---|---|---|
| Unary RPC | Uma mensagem | Uma mensagem | Requisição-resposta tradicional |
| Server Streaming | Uma mensagem | Stream | Cliente envia, servidor responde com stream |
| Client Streaming | Stream | Uma mensagem | Cliente envia stream, servidor responde uma vez |
| Bidirectional Streaming | Stream | Stream | Ambos trafegam streams simultaneamente |

## 6.2 Definição do Contrato (.proto)

O contrato do serviço é definido em linguagem Protocol Buffers (proto3). O compilador protoc gera código cliente e servidor para múltiplas linguagens a partir do mesmo arquivo .proto:

```
  Protocol Buffers — sensor_service.proto
// arquivo: sensor_service.proto
syntax = "proto3";

package sensor.v1;

option java_package = "com.exemplo.sensor.v1";
option java_multiple_files = true;

service SensorService {
  // Unary: solicita leitura única de sensor
  rpc ObterLeitura (SensorRequest) returns (SensorResponse);

  // Server Streaming: servidor envia leituras contínuas
  rpc MonitorarSensor (SensorRequest) returns (stream SensorResponse);
```

```
  // Client Streaming: cliente envia batch de leituras
  rpc EnviarLoteReadings (stream SensorRequest) returns (SummaryResponse);

  // Bidirectional Streaming: chat bidirecional de dados
  rpc StreamBidirecional (stream SensorRequest) returns (stream
SensorResponse);
}

message SensorRequest {
  string sensor_id = 1;
  int64 timestamp = 2;
}

message SensorResponse {
  string sensor_id = 1;
  double temperatura = 2;
  double umidade = 3;
  int64 timestamp = 4;
}

message SummaryResponse {
  int32 total_recebido = 1;
  double media_temperatura = 2;
}
```

## 6.3 Implementação em Java

```
  Java — gRPC Server Streaming & Client Streaming
// pom.xml: grpc-spring-boot-starter, grpc-stub, protobuf-java

import io.grpc.stub.StreamObserver;
import net.devh.boot.grpc.server.service.GrpcService;

@GrpcService
public class SensorServiceImpl extends
SensorServiceGrpc.SensorServiceImplBase {

    /**
     * Server Streaming: monitora sensor continuamente.
     * O servidor emite leituras a cada segundo enquanto o cliente estiver
conectado.
     */
    @Override
    public void monitorarSensor(SensorRequest req,
                                StreamObserver<SensorResponse>
responseObserver) {
        try {
            for (int i = 0; i < 100; i++) {
                if (Thread.currentThread().isInterrupted()) break;

                SensorResponse leitura = SensorResponse.newBuilder()
                    .setSensorId(req.getSensorId())
                    .setTemperatura(20.0 + Math.sin(i) * 5.0)
                    .setUmidade(60.0 + Math.cos(i) * 10.0)
                    .setTimestamp(System.currentTimeMillis())
```

```
                    .build();

                responseObserver.onNext(leitura); // emite um evento
                Thread.sleep(1000);
            }
            responseObserver.onCompleted(); // sinaliza fim do stream

        } catch (InterruptedException e) {
            responseObserver.onError(e);
        }
    }

    /**
     * Client Streaming: recebe batch de leituras do cliente,
     * retorna um resumo ao final.
     */
    @Override
    public StreamObserver<SensorRequest> enviarLoteReadings(
            StreamObserver<SummaryResponse> responseObserver) {

        return new StreamObserver<SensorRequest>() {
            int total = 0;

            @Override
            public void onNext(SensorRequest req) {
                total++;
                System.out.println("Recebido: " + req.getSensorId());
            }

            @Override
            public void onError(Throwable t) {
                System.err.println("Erro no stream: " + t.getMessage());
            }

            @Override
            public void onCompleted() {
                // Envia resumo e encerra
                responseObserver.onNext(SummaryResponse.newBuilder()
                    .setTotalRecebido(total)
                    .build());
                responseObserver.onCompleted();
            }
        };
    }
}
```

## 6.4 Implementação em Python com grpcio

```
  Python — gRPC Streaming com grpcio
# pip install grpcio grpcio-tools
# Gerar código: python -m grpc_tools.protoc --python_out=. --
grpc_python_out=. sensor_service.proto

import grpc, time, math
import sensor_service_pb2 as pb
import sensor_service_pb2_grpc as pb_grpc
```

```
from concurrent import futures


class SensorServiceServicer(pb_grpc.SensorServiceServicer):

    def MonitorarSensor(self, request, context):
        """
        Server Streaming: emite leituras do sensor a cada segundo.
        Verifica context.is_active() para detectar desconexão do cliente.
        """
        i = 0
        while context.is_active():
            yield pb.SensorResponse(
                sensor_id=request.sensor_id,
                temperatura=20.0 + math.sin(i) * 5.0,
                umidade=60.0 + math.cos(i) * 10.0,
                timestamp=int(time.time() * 1000)
            )
            time.sleep(1)
            i += 1

    def EnviarLoteReadings(self, request_iterator, context):
        """
        Client Streaming: processa batch do cliente, retorna resumo.
        """
        total = 0
        soma_temp = 0.0

        for req in request_iterator:
            total += 1
            # Simula processamento
            soma_temp += 20.0 + hash(req.sensor_id) % 10

        media = soma_temp / total if total > 0 else 0.0
        return pb.SummaryResponse(total_recebido=total,
media_temperatura=media)


def iniciar_servidor():
    servidor = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
    pb_grpc.add_SensorServiceServicer_to_server(SensorServiceServicer(),
servidor)
    servidor.add_insecure_port("[::]:50051")
    servidor.start()
    print("Servidor gRPC em :50051")
    servidor.wait_for_termination()


# ─── Cliente gRPC em Python ──────────────────────────────────────────
def cliente_server_streaming():
    with grpc.insecure_channel("localhost:50051") as channel:
        stub = pb_grpc.SensorServiceStub(channel)
        request = pb.SensorRequest(sensor_id="S-01")

        print("Monitorando sensor S-01...")
        for leitura in stub.MonitorarSensor(request):
            print(f"T={leitura.temperatura:.1f}°C  U={leitura.umidade:.1f}%")
```

```
if __name__ == "__main__":
    iniciar_servidor()
```

**gRPC-Web**

O protocolo gRPC utiliza HTTP/2 com trailers (trailers HTTP), que não são suportados nativamente pelos navegadores web. Para expor gRPC para browsers, utiliza-se gRPC-Web (com um proxy intermediário como Envoy) ou conectar-rpc, que adapta o protocolo para funcionar sobre HTTP/1.1 e navegadores.

# 7. Reactive Streams (Project Reactor)

## 7. Reactive Streams e Programação Reativa

A especificação Reactive Streams (definida em reactive-streams.org) padroniza o processamento assíncrono de streams com controle de backpressure. O objetivo é prevenir que um produtor de dados sobrecarregue um consumidor mais lento, um problema crítico em pipelines de streaming de alto throughput.

### 7.1 Componentes da Especificação Reactive Streams

A especificação define quatro interfaces fundamentais:

- Publisher<T>: fonte de dados que emite sequências de elementos do tipo T. Possui o método subscribe(Subscriber).
- Subscriber<T>: consumidor que implementa onSubscribe, onNext, onError e onComplete.
- Subscription: representa a ligação entre Publisher e Subscriber; permite solicitar elementos (request) e cancelar a assinatura (cancel).
- Processor<T,R>: combina Publisher e Subscriber; transforma elementos de tipo T em tipo R.

### 7.2 Backpressure: O Problema Central

Backpressure é o mecanismo pelo qual um Subscriber comunica ao Publisher a sua capacidade de processar elementos. Sem backpressure, um Publisher rápido pode saturar a memória do Subscriber. O protocolo Reactive Streams resolve isso com o método Subscription.request(n), no qual o Subscriber solicita exatamente n elementos por vez.

> **Analogia de Backpressure**
> Imagine uma torneira (Publisher) conectada a um copo (Subscriber). Sem backpressure, a torneira jorra água na máxima velocidade e o copo transborda. Com backpressure, o copo sinaliza quando está pronto para mais água — a torneira se ajusta à capacidade do receptor.

### 7.3 Project Reactor: Mono e Flux

O Project Reactor é a implementação reativa do Spring, que provê dois tipos principais: Mono<T> (0 ou 1 elemento) e Flux<T> (0 a N elementos, potencialmente infinito). Ambos implementam a especificação Reactive Streams e integram-se nativamente com Spring WebFlux.

**Java — Project Reactor: Flux e Backpressure**

```java
// pom.xml: reactor-core

import reactor.core.publisher.*;
import reactor.core.scheduler.Schedulers;
import java.time.*;
import java.util.List;

public class ReactiveStreamsDemos {

    // ─── Criação de Flux ──────────────────────────────────────────
    public void exemploCriacaoFlux() {
        // Flux estático
        Flux<Integer> numeros = Flux.range(1, 100);

        // Flux de intervalo (infinito)
        Flux<Long> ticks = Flux.interval(Duration.ofSeconds(1));

        // Flux programático com Sink
        Flux<String> customFlux = Flux.create(sink -> {
            for (int i = 0; i < 5; i++) {
                sink.next("item-" + i);
            }
            sink.complete();
        });

        // Mono assíncrono
        Mono<String> asyncMono = Mono.fromCallable(() -> {
            Thread.sleep(100); // operação bloqueante
            return "resultado";
        }).subscribeOn(Schedulers.boundedElastic());
    }

    // ─── Operadores de Transformação ──────────────────────────────
    public void exemploOperadores() {
        Flux.range(1, 20)
            .filter(n -> n % 2 == 0)        // apenas pares
            .map(n -> "item-" + n)           // transforma para String
            .take(5)                         // limita a 5 elementos
            .delayElements(Duration.ofMillis(100)) // adiciona delay
            .subscribe(
                item  -> System.out.println("Recebido: " + item),
                error -> System.err.println("Erro: " + error),
                ()    -> System.out.println("Concluído!")
            );
    }

    // ─── Backpressure Explícito ───────────────────────────────────
    public void exemploBackpressure() {
        Flux.range(1, 1000)
            .onBackpressureBuffer(50) // buffer de 50 se consumidor for lento
```

```
            .publishOn(Schedulers.boundedElastic())
            .subscribe(new BaseSubscriber<Integer>() {
                @Override
                protected void hookOnSubscribe(Subscription subscription) {
                    request(10); // solicita apenas 10 elementos inicialmente
                }

                @Override
                protected void hookOnNext(Integer value) {
                    processarItem(value);
                    if (value % 10 == 0) {
                        request(10); // solicita mais 10 após processar o
lote
                    }
                }

                private void processarItem(int v) {
                    System.out.println("Processando: " + v);
                }
            });
    }

    // ─── flatMap e Concorrência ──────────────────────────────────────
    public void exemploFlatMap() {
        // flatMap: processa cada elemento concorrentemente
        Flux.range(1, 10)
            .flatMap(id ->
                Mono.fromCallable(() -> buscarDados(id))
                    .subscribeOn(Schedulers.boundedElastic()),
                4 // concorrência máxima: 4 chamadas simultâneas
            )
            .subscribe(System.out::println);
    }

    private String buscarDados(int id) { return "data-" + id; }
}
```

## 7.4 Integração com Spring WebFlux para Streaming HTTP

```
  Java — Reactive Streaming para HTTP com WebFlux
import org.springframework.http.MediaType;
import org.springframework.web.bind.annotation.*;
import reactor.core.publisher.Flux;
import java.time.*;

@RestController
@RequestMapping("/api/reactive")
public class ReactiveStreamController {

    /**
     * Pipeline reativo completo:
     * 1. Gera IDs a cada 500ms
     * 2. Busca dados para cada ID (simulado)
     * 3. Filtra resultados inválidos
     * 4. Transmite como SSE via text/event-stream
     */
```

```
    @GetMapping(value = "/pipeline", produces =
MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<String> streamPipeline() {
        return Flux.interval(Duration.ofMillis(500))
            .take(50)                           // limita a 50 eventos
            .flatMap(id ->
                Mono.fromCallable(() -> gerarDados(id))
                    .subscribeOn(Schedulers.boundedElastic()),
                3                               // concorrência = 3
            )
            .filter(dado -> dado != null)
            .map(dado -> "data: " + dado)
            .onErrorContinue((err, val) ->
                System.err.println("Erro ignorado: " + err));
    }

    private String gerarDados(long id) {
        return String.format('{"id":%d,"val":%.1f}',
            id, Math.random() * 100);
    }
}
```

## 7.5 Reactive Streams em Python com RxPY e Async Generators

Em Python, o conceito de backpressure e fluxo reativo pode ser implementado através de geradores assíncronos nativos (PEP 525) ou com a biblioteca RxPY:

```
  Python — Reactive Streams com Geradores Assíncronos
# pip install rx aioreactive

# ─── Geradores Assíncronos (equivalente ao Flux) ──────────────────────
import asyncio
from typing import AsyncIterator

async def produtor_de_dados(total: int, delay: float = 0.1) ->
AsyncIterator[dict]:
    """
    Gerador assíncrono como alternativa pythônica ao Flux.
    O loop de eventos controla a execução, fornecendo backpressure natural.
    """
    for i in range(total):
        await asyncio.sleep(delay)  # simula operação assíncrona (I/O)
        yield {"id": i, "valor": i ** 2}


async def consumidor_com_backpressure():
    """
    O consumidor controla o ritmo de consumo implicitamente:
    o produtor só avança quando o consumidor faz "await" no próximo item.
    """
    async for item in produtor_de_dados(100, delay=0.05):
        # Processamento que pode ser mais lento que a produção
        await asyncio.sleep(0.1)  # 2x mais lento que o produtor
        print(f"Processado: {item}")
```

```python
# ─── Pipeline com operadores estilo RxPY ──────────────────────────
from rx import create, operators as ops
from rx.scheduler import ThreadPoolScheduler
import multiprocessing

def pipeline_rx():
    pool_scheduler = ThreadPoolScheduler(multiprocessing.cpu_count())

    source = create(lambda obs, _: [
        (obs.on_next(i), None)[0] for i in range(100) or obs.on_completed()
    )

    source.pipe(
        ops.filter(lambda x: x % 2 == 0),
        ops.map(lambda x: x * x),
        ops.take(10),
        ops.observe_on(pool_scheduler),
    ).subscribe(
        on_next=lambda x: print("RxPY:", x),
        on_error=lambda e: print("Erro:", e),
        on_completed=lambda: print("Completo!")
    )


# ─── Queue com controle de backpressure explícito ─────────────────
async def pipeline_com_queue():
    """
    asyncio.Queue como mecanismo de backpressure:
    maxsize limita quantos itens podem ser enfileirados.
    """
    fila = asyncio.Queue(maxsize=10)  # buffer máximo de 10

    async def produtor():
        for i in range(100):
            await fila.put(i)  # bloqueia se a fila estiver cheia
            print(f"Produzido: {i} (fila: {fila.qsize()}/10)")

    async def consumidor():
        while True:
            item = await fila.get()
            await asyncio.sleep(0.15)  # processa mais lentamente
            fila.task_done()
            print(f"Consumido: {item}")
            if item == 99: break

    await asyncio.gather(produtor(), consumidor())

asyncio.run(pipeline_com_queue())
```

# 8. Comparação e Casos de Uso

# 8. Comparação Aprofundada e Guia de Escolha

## 8.1 Tabela Comparativa Completa

| Critério | Long Polling | SSE | WebSocket | gRPC Streaming |
|---|---|---|---|---|
| Protocolo | HTTP/1.1 | HTTP/1.1+ | WS (sobre TCP) | HTTP/2 |
| Direção | Servidor → Cliente | Servidor → Cliente | Bidirecional | Bidirecional |
| Suporte no Browser | Universal | Nativo (EventSource) | Nativo | Via gRPC-Web |
| Reconexão Automática | Manual | Automática | Manual | Manual |
| Overhead por Mensagem | Alto (HTTP headers) | Baixo | Muito baixo | Muito baixo |
| Multiplexação | Não | Não (HTTP/1.1) | Não nativo | Nativa (HTTP/2) |
| Compressão | HTTP gzip | HTTP gzip | Extensão | gzip / snappy |
| Tipagem forte | Não | Não | Não | Sim (Protobuf) |
| Complexidade de impl. | Baixa | Baixa | Média | Alta |
| Escalabilidade horizontal | Simples | Simples | Requer sticky session | Alta |

## 8.2 Guia de Decisão por Cenário

### Dashboards e Monitoramento em Tempo Real

Recomendação: SSE. Dashboards de monitoramento tipicamente exigem fluxo unidirecional (servidor → cliente), onde o servidor envia atualizações de métricas, alertas e status periodicamente. O SSE é ideal por sua simplicidade, reconexão automática e compatibilidade nativa com navegadores.

### Chat e Aplicações Colaborativas

Recomendação: WebSocket. Comunicação bidirecional em tempo real, com latência mínima, é essencial para aplicações de chat, edição colaborativa (como Google Docs) e jogos multiplayer. WebSocket provê o canal full-duplex necessário com overhead mínimo por mensagem.

### Comunicação entre Microserviços

Recomendação: gRPC Streaming. Para comunicação interna entre serviços, gRPC oferece tipagem forte com protobuf, multiplexação HTTP/2, streaming bidirecional eficiente e suporte a múltiplas linguagens a partir do mesmo contrato .proto.

### Exportação de Grandes Volumes de Dados

Recomendação: HTTP Chunked Transfer. Para exportar relatórios, dumps de banco de dados ou grandes arquivos, o chunked transfer encoding permite streaming progressivo sem buffering total em memória, sem a necessidade de protocolos especializados.

### Notificações Push Simples

Recomendação: SSE ou Long Polling. Para notificações push de baixa frequência (por exemplo, alertas de e-mail, atualizações de sistema), SSE é a escolha moderna. Long Polling é uma alternativa quando compatibilidade com infraestrutura legada for necessária.

## 8.3 Padrões Arquiteturais para Streaming Escalável

### 8.3.1 Fan-out com Message Broker

Para sistemas com múltiplos clientes conectados, o servidor não deve manter estado de mensagens em memória. O padrão recomendado utiliza um message broker (Redis Pub/Sub, Apache Kafka, RabbitMQ) como intermediário:

**Java — SSE Escalável com Redis Pub/Sub**

```
// Java: SSE com Redis Pub/Sub (Lettuce + Spring WebFlux)

import io.lettuce.core.*;
import io.lettuce.core.pubsub.*;
import io.lettuce.core.pubsub.api.reactive.*;
import org.springframework.web.bind.annotation.*;
import org.springframework.http.MediaType;
import org.springframework.http.codec.ServerSentEvent;
import reactor.core.publisher.Flux;

@RestController
```

```
public class ScalableSSEController {

    private final RedisPubSubReactiveCommands<String, String> pubSub;

    public ScalableSSEController(RedisClient redisClient) {
        this.pubSub = redisClient.connectPubSub().reactive();
    }

    /**
     * Qualquer instância do serviço pode servir este endpoint.
     * Os eventos chegam via Redis Pub/Sub — escalável horizontalmente.
     */
    @GetMapping(value = "/api/notificacoes/{canal}",
                produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<ServerSentEvent<String>> stream(@PathVariable String canal) {
        return pubSub.subscribe(canal)
            .thenMany(pubSub.observeChannels())
            .filter(msg -> msg.getChannel().equals(canal))
            .map(msg -> ServerSentEvent.<String>builder()
                .event("notificacao")
                .data(msg.getMessage())
                .build());
    }
}
```

### 8.3.2 Reconexão com Estado via Last-Event-ID

```
  Python — SSE com Recuperação de Eventos Perdidos
# Python: SSE com reenvio de eventos perdidos

from fastapi import FastAPI, Request
from sse_starlette.sse import EventSourceResponse
from collections import deque
import json, asyncio

app = FastAPI()

# Buffer circular dos últimos 100 eventos
event_store: deque = deque(maxlen=100)
event_counter = 0


@app.get("/api/sse/com-recuperacao")
async def sse_com_recuperacao(request: Request,
                              lastEventId: str = None):
    """
    Ao reconectar com Last-Event-ID, reexibe eventos perdidos.
    """
    async def gerar():
        # Reenvia eventos perdidos após reconexão
        if lastEventId:
            ultimo_id = int(lastEventId)
            eventos_perdidos = [
                e for e in event_store if e["id"] > ultimo_id
            ]
            for evento in eventos_perdidos:
                yield {"id": str(evento["id"]), "data": json.dumps(evento)}
```

```
        # Continua com novos eventos
        while not await request.is_disconnected():
            global event_counter
            event_counter += 1
            evento = {"id": event_counter, "msg": f"evento-{event_counter}"}
            event_store.append(evento)
            yield {"id": str(event_counter), "data": json.dumps(evento)}
            await asyncio.sleep(1)

    return EventSourceResponse(gerar())
```

## 9. Considerações para Produção

# 9. Considerações para Ambientes de Produção

## 9.1 Segurança

- TLS/HTTPS obrigatório: todos os protocolos de streaming devem operar sobre TLS. WebSocket sobre TLS usa o esquema wss://, SSE e chunked usam https:// normalmente.
- CORS: para SSE e WebSocket, o servidor deve configurar políticas CORS adequadas, validando a origem da requisição.
- Autenticação: conexões de longa duração devem ser autenticadas. Em WebSocket, os tokens JWT devem ser enviados no handshake (header ou query param). Em SSE, tokens são enviados via header Authorization ou cookies HttpOnly.
- Rate limiting: conexões de streaming consomem recursos por períodos prolongados; implemente limites de conexões simultâneas por usuário/IP.

## 9.2 Gerenciamento de Recursos

- Thread pools: servidores bloqueantes (Spring MVC) alocam uma thread por conexão; com milhares de streams simultâneos, o modelo reativo (Spring WebFlux, FastAPI) é necessário.
- Timeouts de conexão: defina timeouts razoáveis no cliente e servidor para evitar conexões zumbi (conexões tecnicamente abertas mas sem consumidor ativo).
- Heartbeats: envie mensagens periódicas de keepalive para detectar clientes desconectados silenciosamente e liberar recursos.
- Monitoramento: instrumente o número de conexões ativas, throughput (bytes/s, eventos/s) e erros com ferramentas como Micrometer (Java) ou Prometheus/OpenTelemetry.

## 9.3 Configuração de Infraestrutura

**Nginx — Configuração para SSE e WebSocket**

```
# nginx.conf: configuração para SSE e WebSocket

server {
    listen 443 ssl;
    server_name api.exemplo.com;
```

```
    # SSE: desabilita buffering para transmissão imediata
    location /api/sse/ {
        proxy_pass http://backend;
        proxy_buffering off;
        proxy_cache off;
        proxy_read_timeout 86400s;     # 24h
        proxy_send_timeout 86400s;
        proxy_set_header Connection '';
        proxy_http_version 1.1;
        add_header X-Accel-Buffering no;
        add_header Cache-Control no-cache;
    }

    # WebSocket: upgrade de protocolo
    location /ws/ {
        proxy_pass http://backend;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_read_timeout 3600s;
        proxy_send_timeout 3600s;
    }
}
```

## 9.4 Testes de Streaming

**Python — Teste de Endpoint SSE com pytest**

```
# Python: teste de SSE com pytest e httpx

import pytest, httpx, asyncio
from httpx import AsyncClient


@pytest.mark.asyncio
async def test_sse_emite_eventos():
    """
    Verifica que o endpoint SSE emite eventos no formato correto.
    """
    eventos_recebidos = []

    async with AsyncClient(app=app, base_url="http://test") as client:
        async with client.stream("GET", "/api/sse/temperatura") as response:
            assert response.status_code == 200
            assert "text/event-stream" in response.headers["content-type"]

            async for linha in response.aiter_lines():
                if linha.startswith("data:"):
                    dados = json.loads(linha[5:].strip())
                    eventos_recebidos.append(dados)

                if len(eventos_recebidos) >= 3:
                    break

    assert len(eventos_recebidos) == 3
```

```
    assert all("graus" in e for e in eventos_recebidos)
```

# 10. Conclusão

O streaming de dados representa um paradigma fundamental para aplicações web modernas que demandam baixa latência, eficiência de recursos e comunicação em tempo real. Cada técnica abordada atende a casos de uso distintos, e a escolha adequada depende de uma análise cuidadosa de requisitos.

O Chunked Transfer Encoding é a solução mais simples para streaming de respostas HTTP, ideal para exportação progressiva de dados sem protocolos especializados. O Long Polling, embora tecnicamente simples, deve ser evitado em novos sistemas em favor do SSE, que oferece reconexão automática e menor overhead. O WebSocket é a escolha natural para comunicação bidirecional em tempo real no contexto web, enquanto o gRPC Streaming é superior para comunicação eficiente e tipada entre microserviços. A programação reativa com Project Reactor abstrai e padroniza o tratamento de backpressure e composição de pipelines de dados assíncronos.

O domínio dessas técnicas, em conjunto com boas práticas de segurança, gerenciamento de recursos e arquitetura escalável, é essencial para a construção de sistemas web robustos e responsivos.

**Próximos Passos Sugeridos**

Para aprofundar o estudo, recomenda-se: (1) implementar um dashboard de monitoramento em tempo real combinando SSE e gráficos Chart.js; (2) construir um sistema de chat com WebSocket, Redis Pub/Sub e autenticação JWT; (3) criar um pipeline de processamento de dados com Kafka + Spring WebFlux; (4) explorar a especificação Reactive Streams completa em reactive-streams.org.